BÁO CÁO KỸ THUẬT – UEBA BENCHMARK

Benchmark Isolation Forest, LOF, One-Class SVM
trên log xác thực Windows (Event 4624/4625)

Thành viên: Lâm Vũ – Đinh Xuân Hiệp | Báo cáo tổng tuần 1–6, ngày 15/10/2026

1. Phân công

Tuần	Lâm Vũ	Đinh Xuân Hiệp	Cả hai
1	Khám phá dữ liệu event 4624, làm data dictionary và value dictionary, viết báo cáo	Khám phá dữ liệu event 4625, vẽ đồ thị mô tả	Tìm và chọn bộ dữ liệu, lọc các event cần thiết
2	Chuẩn hoá khoá và phân tầng User/Machine, thiết kế 18 đặc trưng, tổng hợp báo cáo	Thực hiện 5 nhóm kiểm định, dựng biểu đồ và slide	Đối chiếu định nghĩa đặc trưng với log 4624/4625, rà soát kết quả
3	Làm giao diện chung và wrapper cho 3 thuật toán, dựng pipeline benchmark, viết báo cáo	Thống kê lần chạy thử đầu, review khung benchmark	–
4	Bổ sung đặc trưng, xây dựng quy trình tiêm, chạy benchmark đánh giá	Kiểm tra lại quy trình tiêm và benchmark, làm slide, báo cáo	–
5	Thực hiện SHAP và ablation	Kiểm tra lại kết quả	–

Mục lục

1. Phân công **1**

Tóm tắt **2**

2. Dữ liệu **3**
2.1. Nguồn . . . 3
2.2. Phân loại tài khoản . . . 3
2.3. Chất lượng dữ liệu và các quyết định làm sạch . . . 3
2.4. Chu kì theo thời gian . . . 4

3. Phương pháp **4**
3.1. Đặc trưng . . . 4
3.2. Mô hình . . . 5
3.3. Mốc tham chiếu . . . 5
3.4. Thiết kế đánh giá . . . 5

4. Kết quả **8**
4.1. Lưới cấu hình trên dev (22 đặc trưng) . . . 8
4.2. Benchmark cuối trên test . . . 8
4.3. Đường PR và độ nhạy contamination . . . 9
4.4. Theo loại tấn công . . . 11
4.5. Chi phí tính toán . . . 12

5. Phân tích **12**
5.1. Vì sao OCSVM vượt LOF . . . 12
5.2. Scaler: lựa chọn “an toàn” theo sách giáo khoa có thể sai . . . 12
5.3. Chọn đặc trưng: từ 41 xuống 22 . . . 13
5.4. Cửa sổ lịch sử 7 / 14 / 21 ngày . . . 15
5.5. Độ ổn định . . . 16
5.6. Mức lạc quan của kết quả . . . 16

6. Đánh giá định tính: review thủ công top 50 **16**
6.1. Thiết lập . . . 16
6.2. Trạng thái . . . 16

7. Hạn chế đã biết **17**

8. Khuyến nghị triển khai **17**
8.1. Kiến trúc phân luồng theo loại tài khoản . . . 17
8.2. Cấu hình mô hình . . . 17
8.3. Ngân sách cảnh báo và chi phí vận hành . . . 18
8.4. Vận hành và theo dõi . . . 18

9. Tái lập và vận hành **18**
9.1. Môi trường . . . 18
9.2. Một lệnh . . . 18
9.3. Từng bước . . . 19
9.4. Mô hình cho demo . . . 19
9.5. Trạng thái kiểm chứng . . . 19

10. Đối chiếu tiêu chí hoàn thành (đề cương mục 4.1) **19**

Phụ lục A: 22 đặc trưng của benchmark cuối **20**

Phụ lục B: bản đồ số liệu **22**

Tóm tắt

1. **Bài toán:** Phát hiện hành vi bất thường của tài khoản người dùng từ log đăng nhập Windows (4624 thành công, 4625 thất bại) của bộ LANL Unified Host, 60 ngày, khoảng 943 triệu sự kiện. Đơn vị phân tích là một dòng (tài khoản, ngày). Không có nhãn thật, nên hiệu năng được đo bằng 6 kịch bản tấn công tiêm vào log.
2. **Đặc trưng:** 41 đặc trưng hành vi đầu thông qua phân tích SHAP và thí nghiệm loại bỏ trên dev rút gọn còn 22 đặc trưng cho benchmark cuối.
3. **Thiết kế đánh giá:** Chia theo thời gian: train ngày 1–42, test ngày 43–60; chọn cấu hình trên khối dev (ngày 36–42, fit 1–35) nằm trong train. Mỗi run tiêm đúng 1% số dòng User. 5 seed, báo cáo trung bình ± độ lệch chuẩn.
4. **Kết quả chính (test, 22 đặc trưng, 5 seed):** One-Class SVM tốt nhất: recall top-100 cảnh báo/ngày, PR-AUC, gấp khoảng 3,4 lần recall và 7 lần PR-AUC của luật ngưỡng.
   Thứ hạng ổn định: OCSVM > IF > LOF > z-score > luật ngưỡng > ngẫu nhiên.
5. **Phát hiện quan trọng:**
   - Phát hiện tốt brute-force, đổi loại logon, bùng nổ máy trạm mới; bắt kém ba loại: dời hoạt động sang ban đêm, rải mật khẩu (password spraying) và tài khoản ngủ đông thức dậy, do đặc trưng chỉ tính theo từng tài khoản trong từng ngày.
   - Chọn scaler quyết định mô hình dựa trên khoảng cách: đổi RobustScaler sang StandardScaler nâng PR-AUC OCSVM từ 0,03 lên 0,24 trên dev.
   - Bớt đặc trưng làm mô hình tốt hơn: từ 41 xuống 22 đặc trưng.
6. **Khuyến nghị:** Triển khai OCSVM, chạy song song một luật riêng cho dò/rải mật khẩu; tài khoản dịch vụ, hệ thống và quản trị dùng luật ngưỡng.

2. Dữ liệu

2.1. Nguồn

Bộ LANL Unified Host and Network Dataset: log Windows Security của mạng doanh nghiệp Los Alamos, đã ẩn danh, 60 ngày (bộ gốc dài 90 ngày, mỗi ngày khoảng 400 triệu bản ghi đủ loại; đề tài dùng 60 ngày). Dự án dùng hai loại sự kiện:

Sự kiện	Ý nghĩa	Ví dụ ngày 1
4624	Đăng nhập thành công	14,1 triệu
4625	Đăng nhập thất bại	0,34 triệu (2,3%)

Mỗi sự kiện có 21 trường, chia 6 nhóm: thời gian (`Time`, số giây tính từ đầu bộ dữ liệu), sự kiện (`EventID`), danh tính (`UserName`, `DomainName`, `LogonID`, `Subject*`), máy (`LogHost`, `Source`, `Destination`), xác thực (`LogonType`, `AuthenticationPackage`, `FailureReason`, `Status`, ...) và tiến trình (`Process*`).

Quy mô sau làm sạch: khoảng 943 triệu sự kiện; ma trận đặc trưng có 1.055.283 dòng (tài khoản, ngày).

2.2. Phân loại tài khoản

Khoá của một tài khoản là cặp (`DomainName`, `UserName`). Tài khoản được chia 5 loại theo tên:

Loại	Dòng train (ngày 1–42)	Dòng test (ngày 43–60)	Số tài khoản (60 ngày)	Tỷ lệ sự kiện
Machine (tên kết thúc `$`)	404.907	180.092	13.497	45%
User	314.847	152.740	18.003	36%
Admin (`Administrator`)	1.371	630	825	3%
Service (`AppService`, `Scanner`, ...)	317	137	10	12%
System (`nt authority\{}...`)	170	72	6	3%

Quyết định: Chỉ thực hiện đánh giá trên nhóm **User** vì:

- **Admin, Service, System** có quá ít thực thể để học máy có ý nghĩa thống kê (ví dụ: System chỉ có 6 tài khoản) và hành vi gần như tất định, phù hợp hơn với tiếp cận dựa trên luật.
- Tài khoản **Machine** đủ mẫu nhưng không phù hợp với kịch bản đã cho.

2.3. Chất lượng dữ liệu và các quyết định làm sạch

Khảo sát chất lượng làm trong tuần 1 phát hiện vấn đề sau: `src/data/cleaner.py` và có unit test (`tests/test_cleaner.py`):

#	Vấn đề phát hiện	Quyết định	Lý do
1	`LogonTypeDescription` thiếu ở vài bản ghi 4624	Điền theo bảng ánh xạ chuẩn của Microsoft từ `LogonType`	Quan hệ 1–1, không cần đoán
2	Ở 4624, `Process*` thiếu 92%, `Subject*` thiếu 99,9%; `Status`, `Destination`, `ServiceName`, `ParentProcess*` trống hoàn toàn ở cả hai sự kiện	Bỏ cột (21 → 11 cột)	Không mang thông tin; giữ lại chỉ tốn bộ nhớ
3	`Source`, `LogonID`, `DomainName` có giá trị thiếu (`Source` thiếu 24% ở 4624 và 32% ở 4625; `LogonID` thiếu 49% ở 4625)	Điền nhãn `"Unknown"`, chuẩn hoá `DomainName` về chữ thường	Giữ đủ số dòng và trục thời gian; thiếu `Source` vẫn là thông tin (có đặc trưng `missing_source_ratio`)
4	Bản ghi trùng hoàn toàn (ngày 1: 2,1%)	Xoá trùng tuyệt đối	Tránh thổi phồng số sự kiện
5	Lệch giờ mùa hè từ ngày 42	Trừ 3.600 giây cho ngày ≥ 42	Đưa 60 ngày về cùng trục 24 giờ, quan trọng cho đặc trưng ngoài giờ

2.4. Chu kì theo thời gian

`Time` được tính bằng giây , từ đó tính ra giờ trong ngày. Bảng dưới kiểm tra giả định đó và các chu kì khác của dữ liệu.

Bảng 2.1: Khảo sát thời gian của 60 ngày dữ liệu

Câu hỏi	Kết quả	Kết luận
Có chu kỳ 7 ngày không?	Hệ số tự tương quan của lưu lượng ngày ở độ trễ 7 là 0,58, cao nhất trong các độ trễ 1–14. Riêng thứ tự ngày trong tuần giải thích 67% biến động lưu lượng	Có
Có "cuối tuần" không?	2 trong 7 ngày có lưu lượng thấp hơn 29% và số tài khoản hoạt động thấp hơn 31%, lặp lại ở 87,5% số tuần (16 ngày: 6, 7, 13, 14, . . . , 55, 56)	Có
Ban đêm có vắng hơn ban ngày không?	Ngày thường, 0h–5h trung bình 553.600 sự kiện 4624/giờ, thấp hơn 37% so với 9h–15h (877.500/giờ). Thấp nhất lúc 4h, cao nhất lúc 9h. Cuối tuần gần như phẳng	Có. Cách tính giờ khớp với giờ địa phương

3. Phương pháp

3.1. Đặc trưng

3.1.1. Quy trình xây bộ 41 đặc trưng

Bộ đặc trưng đi qua 4 phiên bản: 18 ở tuần 2 (16 tính trong ngày, 2 lịch sử) → 24 ở tuần 3 → 39 → 41 ở tuần 4. Phiên bản v4 dùng không gian tổ hợp 4 trục để sinh ứng viên một cách có hệ thống thay vì liệt kê theo cảm tính:

Trục	Giá trị
Phép đo (14)	đếm, đếm phân biệt, tỷ phần, độ đều, fan-out, độ mới, Jaccard, độ dịch phân bố, chênh so với trung bình, z so với nhóm, . . .
Đối tượng (4)	mọi sự kiện, thất bại, thành công, ban đêm
Chiều (8)	sự kiện, Source, LogHost, cặp (Source, LogHost), giờ, loại logon, gói xác thực, lý do thất bại
Cửa sổ (7)	1 ngày; khung 5/15/60 phút; 7/14 ngày trước; toàn lịch sử

3.136 tổ hợp được tạo ra sẽ trải qua quá trình lọc về ngữ nghĩa, rò rỉ thời gian, độ lớn VIF, có null nhiều hay không, khả năng phân tách kịch bản. Cuối cùng chốt được 41 đặc trưng

3.1.2. Bộ 22 đặc trưng của benchmark cuối

Trên khối dev, SHAP cho thấy nhóm rhythm (nhịp thời gian giữa các sự kiện) ít giúp bắt nạn nhân nhưng là nguồn báo nhầm chính của OCSVM, và 18 đặc trưng có đóng góp dưới mức chia đều ở mọi mô hình, mọi run. Sau đó tiến hành loại bỏ dần dần và kiểm tra trên test ở mục 5.3 và chốt được bộ đặc trưng tốt nhất là bộ 22 đặc trưng.

3.2. Mô hình

Ba mô hình và bốn mốc tham chiếu dùng chung một giao diện (`src/models/base.py`, `pipeline.py`): đặc trưng → điền thiếu bằng median → scaler → mô hình, mọi bước chỉ fit trên train của phân khúc; điểm càng cao càng bất thường; ngưỡng cảnh báo = phân vị (1 − contamination) của điểm train.

Mô hình	Cài đặt	Lý do
Isolation Forest	scikit-learn `IsolationForest`, fit toàn bộ train	Chuẩn, nhanh, không phụ thuộc thang đo
LOF	LOF trên láng giềng xấp xỉ HNSW, chỉ mục trên các dòng duy nhất	LOF chính xác không chạy được trên 315.000 dòng × nhiều cấu hình; khử trùng tránh k-distance = 0 (điểm LOF $\approx 10^{10}$)
One-Class SVM	Nhân RBF xấp xỉ Nystroem + `SGDOneClassSVM`	OCSVM chính xác có độ phức tạp bậc 2–3 theo số mẫu; `nu` = contamination

Tham số mặc định ở `configs/model_params.yaml`; các cấu hình của lưới ở `configs/benchmark_grid_dev_22f.yaml`.

3.3. Mốc tham chiếu

Mốc	Điểm số	Vai trò
Ngẫu nhiên	Băm tất định theo dòng	Sàn hiệu năng (đề cương 5.3)
Luật số lần thất bại ≥ 5	Số lần thất bại trong ngày; ngưỡng cố định L = 5 ước lượng từ log train (trung vị số lần sai trước khi bị khoá)	Thứ đội vận hành làm được trong 10 phút (đề cương 5.3)
Luật 6 điều kiện	Tỷ lệ luật thoả: thất bại, khoá, NTLM, ngoài giờ, cùng giây, từ xa	Luật phong phú hơn
Z-score bền vững	max |z| (median, MAD) trên các đặc trưng	Đo giá trị của mô hình hoá đa biến

Z-score lấy max trên nhiều đặc trưng thay vì một đặc trưng như đề cương mô tả, vì không có một đặc trưng đơn lẻ nào đại diện cho cả 6 kịch bản. Chi tiết: `reports/week4/baselines_report.md`.

3.4. Thiết kế đánh giá

3.4.1. Chia tập theo thời gian

Khối	Fit	Chấm	Dùng để
Dev	ngày 1–35	ngày 36–42	Chọn cấu hình, chọn đặc trưng, SHAP
Test	ngày 1–42 (70% thời gian)	ngày 43–60 (30%)	Báo cáo kết quả, chạy với cấu hình đã chốt

Dev nằm hoàn toàn trong train, test không tham gia bất kỳ lựa chọn nào (một ngoại lệ ghi ở mục 5.3).

3.4.2. Nhãn tổng hợp: 6 kịch bản tấn công

Bộ tiêm (`src/injection/`) chèn sự kiện vào log, sau đó toàn bộ đặc trưng được tính lại, kể cả đặc trưng lịch sử 7 ngày. Nguyên tắc: không bịa giá trị trường, mọi sự kiện tiêm là bản sao một sự kiện thật của train (của chính nạn nhân nếu có); hồ sơ hành vi chỉ dựng từ train của khối; nạn nhân là tài khoản User có ≥ 7 ngày hoạt động trong train, mỗi tài khoản tối đa một lần; bỏ các (tài khoản, ngày) vốn đã bị luật cảnh báo trên log gốc.

Kịch bản	Hành vi được tiêm	Đề cương 5.2
`brute_force`	8–25 lần 4625 từ một nguồn lạ trong 5–30 phút; bị khoá sau L = 5 lần sai	Có
`password_spraying`	Một nguồn lạ thử 1–3 lần sai trên 5–13 tài khoản trong một cửa sổ 30–90 phút	Có
`off_hours`	Dời toàn bộ một ngày hoạt động điển hình sang 0–7 giờ	Có
`new_workstation_burst`	Đăng nhập thành công từ 4–10 máy chưa từng dùng trong 0,5–2 giờ	Có
`dormant_wakeup`	Tài khoản im lặng lâu hơn khoảng nghỉ dài nhất của chính nó bỗng có một ngày hoạt động điển hình	Có
`logon_type_switch`	Cả ngày đăng nhập chuyển sang loại logon chưa từng dùng, ưu tiên 2 $\rightarrow$ 3/5	Một phần (log chỉ có khoảng 35 tài khoản chủ yếu dùng type 2)

Lượng tiêm: mỗi run tiêm đúng 1% số dòng User của khối chấm (đề cương: 0,5–1%): dev 553 nạn nhân/run, test 1.543 nạn nhân/run (257–258 mỗi kịch bản). Mỗi khối có 5 run với 5 seed tiêm khác nhau.

Bộ tiêm đã được đối chiếu với đề cương và sửa 4 chỗ (2 lỗi mã, 2 lệch thiết kế) trước khi chạy lưới chính thức; chi tiết ở `reports/week4/tom_tat_tuan4.md` mục 2.3.

3.4.3. Chỉ số

Chỉ số	Định nghĩa	Vai trò
Recall top-100/ngày	Mỗi ngày lấy 100 dòng điểm cao nhất; tỷ lệ nạn nhân lọt vào, gộp qua các ngày	Chỉ số chính: khớp năng lực xem khoảng 100 cảnh báo/ngày của SOC
PR-AUC	Diện tích dưới đường precision–recall	Chỉ số chính của đề cương; đo chất lượng xếp hạng toàn danh sách
ROC-AUC	Khả năng phân tách tổng thể	Chỉ số phụ; dùng khi so theo kịch bản
P@10/50/100	Tỷ lệ nạn nhân trong K dòng điểm cao nhất của cả khối	Trải nghiệm analyst
Recall tại ngân sách 5%	Recall cảnh báo 5% dòng điểm cao nhất	Đề cương 5.4
Tỷ lệ cảnh báo	Tỷ lệ dòng vượt ngưỡng train	Gánh nặng vận hành
Thời gian fit / chấm	Đo trên cùng máy	Khả năng mở rộng

Recall top-100/ngày được chọn làm chỉ số chính vì ROC-AUC quá lạc quan khi dương chỉ chiếm 1%, còn PR-AUC cộng cả những mức recall analyst không bao giờ đạt tới.

3.4.4. Seed, chọn cấu hình và nhật ký

- Một "seed" = một cặp (run tiêm, `random_state` mô hình): 5 cặp với model seed 42, 7, 2024, 1, 2. Độ lệch chuẩn vì vậy gồm cả biến động của dữ liệu tiêm lẫn của mô hình.
- Quy tắc chọn cấu hình (ghi manifest trước khi chạy): PR-AUC trung bình cao nhất trên dev.
- Dòng train được sắp cố định trước khi fit, vì IF và OCSVM (SGD) phụ thuộc thứ tự dòng.
- Nhật ký chung `experiments/logs/experiment_log.csv`: mọi lần fit ghi thời điểm, commit, cấu hình, run, seed, kết quả (đề cương 5.6). Mỗi lưới có thêm `grid_manifest.json` với commit, phiên bản thư viện và sha256 của ma trận đầu vào.

4. Kết quả

4.1. Lưới cấu hình trên dev (22 đặc trưng)

Bảng 4.1: 3 mô hình × 13 cấu hình, TB ± std qua 5 run dev (cấu hình được chọn in đậm)

Mô hình	Cấu hình	Recall top-100/ngày	PR-AUC
IF	mặc định (100 cây, max_samples auto)	0,238 ± 0,011	0,093 ± 0,010
IF	300 cây, max_samples 1024	0,264 ± 0,012	0,128 ± 0,016
IF	max_features 0,5	0,227 ± 0,036	0,093 ± 0,016
LOF	k = 20, RobustScaler	0,192 ± 0,025	0,078 ± 0,025
LOF	k = 20, StandardScaler	0,060 ± 0,010	0,014 ± 0,001
LOF	k = 20, QuantileTransformer	0,082 ± 0,015	0,027 ± 0,002
LOF	k = 100, StandardScaler	0,084 ± 0,016	0,018 ± 0,002
OCSVM	RobustScaler	0,094 ± 0,026	0,033 ± 0,010
OCSVM	StandardScaler, γ = scale, 300 thành phần	0,412 ± 0,034	0,239 ± 0,026
OCSVM	QuantileTransformer	0,174 ± 0,016	0,073 ± 0,006
OCSVM	StandardScaler, 1.000 thành phần (`nc1000`)	0,383 ± 0,067	0,227 ± 0,049
OCSVM	StandardScaler, γ = 0,005	0,395 ± 0,015	0,215 ± 0,009
OCSVM	StandardScaler, γ = 0,1 (`g01`)	0,407 ± 0,033	0,248 ± 0,028

Ba cấu hình OCSVM StandardScaler hàng đầu chênh nhau dưới 1 std; theo quy tắc chọn (PR-AUC cao nhất), `g01` được chọn.

4.2. Benchmark cuối trên test

Bảng 4.2: TB ± std qua 5 run test, 22 đặc trưng, tỷ lệ dương 1,01% (mỗi run khoảng 153.570 dòng chấm, 1.543 nạn nhân)

Mô hình	PR-AUC	ROC-AUC	P@10	P@50	P@100	Recall top-100/ngày	Recall top 5%	Tỷ lệ cảnh báo	Fit (s)	Chấm (s)
OCSVM (`g01`)	0,175 ± 0,048	0,892 ± 0,032	0,46 ± 0,18	0,38 ± 0,10	0,29 ± 0,10	0,336 ± 0,070	0,606 ± 0,054	0,069 ± 0,005	5 ± 0	0,9 ± 0,1
IF	0,111 ± 0,011	0,921 ± 0,005	0,24 ± 0,17	0,19 ± 0,03	0,22 ± 0,03	0,242 ± 0,016	0,468 ± 0,028	0,067 ± 0,001	12 ± 1	2,2 ± 0,1
LOF	0,071 ± 0,013	0,717 ± 0,007	0,28 ± 0,36	0,11 ± 0,14	0,11 ± 0,10	0,175 ± 0,006	0,360 ± 0,017	0,065 ± 0,001	8 ± 0	2,6 ± 0,1
Z-score	0,032 ± 0,003	0,743 ± 0,003	0,14 ± 0,09	0,25 ± 0,10	0,23 ± 0,04	0,075 ± 0,011	0,154 ± 0,010	0,072 ± 0,001	2 ± 0	0,2 ± 0,0
Luật 6 điều kiện	0,015 ± 0,001	0,536 ± 0,004	0,00 ± 0,00	0,02 ± 0,03	0,04 ± 0,03	0,073 ± 0,012	0,124 ± 0,011	0,361 ± 0,000	1 ± 0	0,1 ± 0,0
Luật thất bại ≥ 5	0,025 ± 0,000	0,623 ± 0,005	0,00 ± 0,00	0,00 ± 0,00	0,01 ± 0,00	0,098 ± 0,010	0,198 ± 0,002	0,050 ± 0,000	1 ± 0	0,1 ± 0,0
Ngẫu nhiên	0,010 ± 0,000	0,503 ± 0,013	0,02 ± 0,04	0,01 ± 0,02	0,02 ± 0,01	0,014 ± 0,003	0,052 ± 0,007	0,050 ± 0,001	2 ± 0	0,3 ± 0,0
OCSVM `nc1000` (đối chiếu)	*0,189 ± 0,013*	*0,908 ± 0,005*	*0,30 ± 0,19*	*0,29 ± 0,16*	*0,25 ± 0,15*	*0,373 ± 0,022*	*0,639 ± 0,022*	*0,074 ± 0,002*	*47 ± 2*	*3,7 ± 0,1*

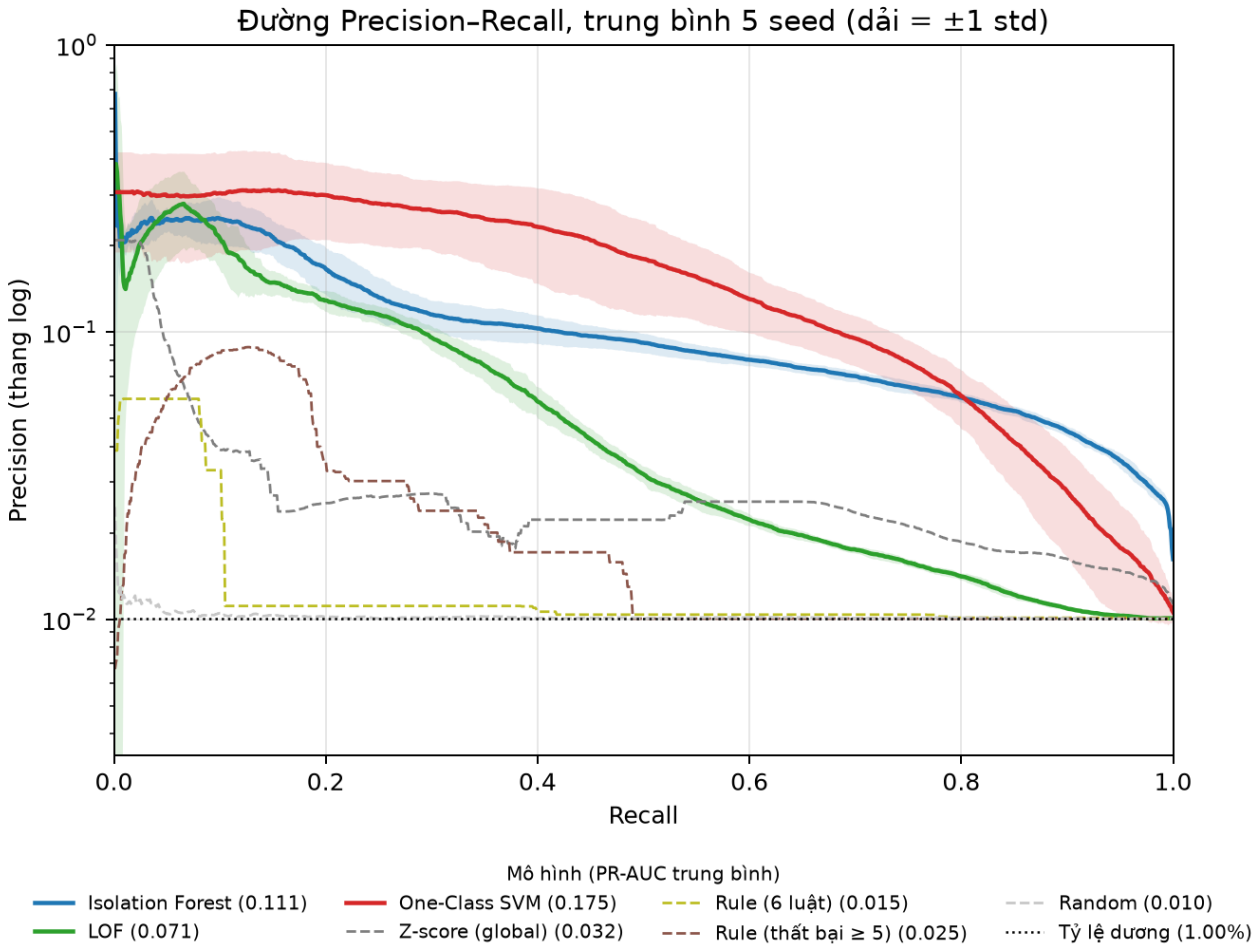
Ghi chú: P@K tính trên cả khối 18 ngày; tỷ lệ cảnh báo ở contamination 5%; luật 6 điều kiện không có ngưỡng điều chỉnh được nên cờ 36% số dòng.

Nhận xét:

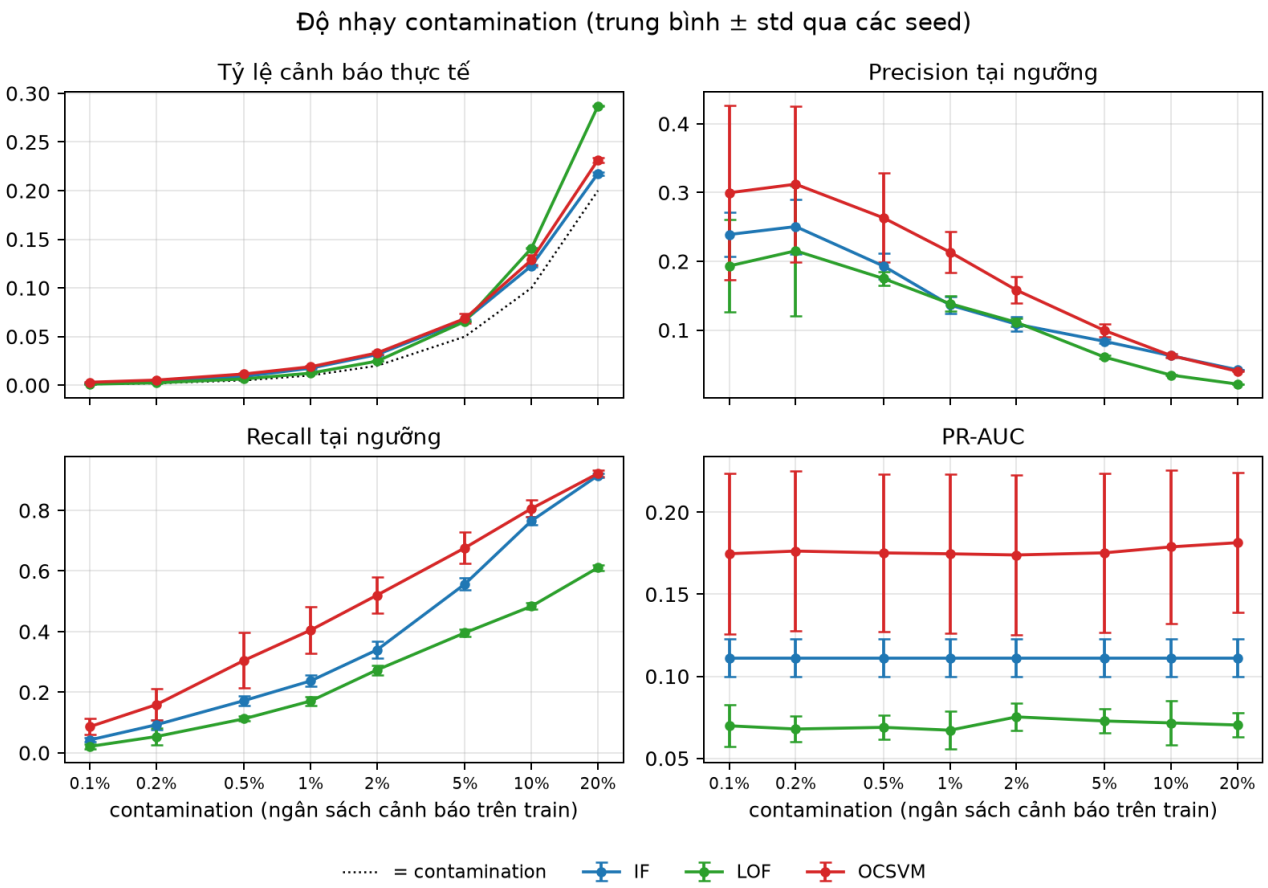
1. OCSVM tốt nhất theo cả chỉ số chính lẫn PR-AUC. Với 100 cảnh báo/ngày, OCSVM bắt khoảng một phần ba số nạn nhân, và khoảng 29% cảnh báo là đúng (precision top-100/ngày 0,29).

2. Cả ba mô hình đều vượt hai mốc của đề cương. So với luật thất bại ≥ 5: OCSVM gấp 3,4 lần về recall top-100/ngày và 7 lần về PR-AUC; IF gấp 2,5 và 4,4 lần; LOF gấp 1,8 và 2,8 lần. So với ngẫu nhiên: PR-AUC gấp 7–17 lần.
3. ROC-AUC không phân biệt được mô hình tốt: IF có ROC-AUC cao nhất (0,921) nhưng recall top-100/ngày chỉ bằng khoảng 70% OCSVM.
4. Mô hình đa biến có giá trị gia tăng thật: z-score (cùng đặc trưng, không mô hình hoá tương quan) chỉ đạt 0,075 recall.
5. Luật thất bại xếp hạng kém (P@10 = 0): những dòng có nhiều lần thất bại nhất là tài khoản thật vốn hay gõ sai, không phải nạn nhân. Vì vậy sinh nhiều báo động giả.

4.3. Đường PR và độ nhạy contamination



Hình 4.1. Đường precision–recall trung bình qua 5 seed. OCSVM giữ precision khoảng 0,3 tới recall khoảng 0,3; z-score chỉ tốt ở vài cảnh báo đầu.



Hình 4.2 và bảng 4.3: Độ nhạy contamination (đề cương 5.6: 0,1–5%, thêm 10% và 20%)

c	OCSVM P / R	IF P / R	LOF P / R	Tỷ lệ cảnh báo thực tế (OCSVM)	Cảnh báo/ngày (OCSVM, khoảng)
0,1%	0,30 / 0,09	0,24 / 0,04	0,19 / 0,02	0,3%	25
0,2%	0,31 / 0,16	0,25 / 0,09	0,22 / 0,05	0,5%	40
0,5%	0,26 / 0,31	0,19 / 0,17	0,18 / 0,11	1,2%	100
1%	0,21 / 0,41	0,14 / 0,24	0,14 / 0,17	1,9%	160
2%	0,16 / 0,52	0,11 / 0,34	0,11 / 0,27	3,3%	280
5%	0,10 / 0,68	0,08 / 0,56	0,06 / 0,40	6,9%	590

- Contamination chỉ dời điểm vận hành, không đổi chất lượng xếp hạng: PR-AUC OCSVM chỉ dao động 0,174–0,181 dù `nu` đổi theo contamination; IF và LOF chỉ đổi ngưỡng.
- Chọn contamination là chọn ngân sách analyst. Ở c = 0,5%, OCSVM bật khoảng 100 cảnh báo/ngày, cứ 4 cảnh báo có 1 tấn công, bắt 31% nạn nhân.
- Tỷ lệ cảnh báo thực tế cao hơn contamination 1,4–2,5 lần do dữ liệu trôi so với train, nên khi triển khai cần đặt ngân sách theo số cảnh báo/ngày.

4.4. Theo loại tấn công

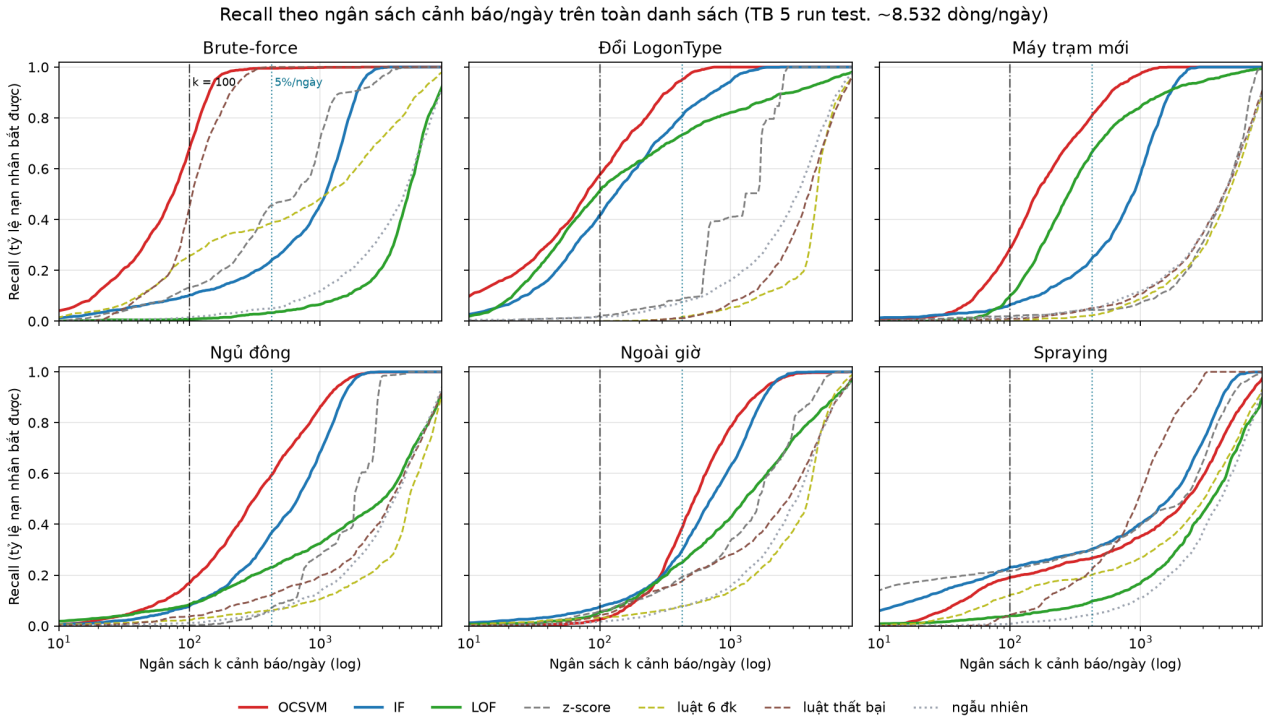
Bảng 4.4: ROC-AUC theo kịch bản, test, 22 đặc trưng, TB 5 run

Kịch bản	OCSVM	IF	LOF	Z-score	Luật thất bại	Luật 6 đk	Ngẫu nhiên
brute_force	0,992	0,945	0,581	0,915	0,988	0,751	0,500
logon_type_ switch	0,973	0,967	0,909	0,791	0,359	0,406	0,501
new_ workstation_ burst	0,973	0,959	0,936	0,521	0,495	0,459	0,491
dormant_wakeup	0,907	0,942	0,554	0,785	0,513	0,418	0,513
off_hours	0,801	0,872	0,739	0,687	0,510	0,571	0,508
password_ spraying	0,709	0,842	0,586	0,759	0,875	0,613	0,503

Bảng 4.5 đo recall top-100/ngày theo kịch bản. Số liệu lấy từ thí nghiệm tuần 5 (41 đặc trưng, OCSVM `nc1000`); thứ hạng kịch bản không đổi với bộ 22.

Bảng 4.5: Recall top-100/ngày theo kịch bản, test, 41 đặc trưng, TB 5 run

Kịch bản	OCSVM	IF	LOF	Z-score	Luật thất bại	Hạng trung vị của nạn nhân (OCSVM)
brute_force	0,68	0,10	0,01	0,13	0,45	top 0,8%
logon_type_switch	0,58	0,42	0,51	0,02	0,00	top 1,2%
new_workstation_burst	0,29	0,07	0,10	0,02	0,01	top 1,8%
password_spraying	0,19	0,23	0,04	0,22	0,05	top 26%
dormant_wakeup	0,17	0,08	0,08	0,00	0,04	top 4,0%
off_hours	0,03	0,08	0,05	0,06	0,04	top 7,7%
Tất cả	0,32	0,16	0,13	0,08	0,10	



Hình 4.3. Recall theo ngân sách k cảnh báo/ngày, mỗi ô một kịch bản (41 đặc trưng). Đường càng lên sớm càng tốt.

Nhận xét:

- Bắt tốt: brute_force (OCSVM gần 0,7 ở 100 cảnh báo/ngày, gần 1,0 ở 5%), logon_type_switch (cả ba mô hình), new_workstation_burst (chủ yếu OCSVM).
- Bắt yếu: dormant_wakeup; nạn nhân bị bỏ sót lại có nhiều sự kiện hơn nạn nhân bị bắt (trung vị 150 so với 85).
- Điểm mù chung: off_hours và password_spraying.
- Không có mô hình nào thắng ở mọi kịch bản: luật thất bại tốt nhất ở spraying về ROC-AUC; IF tốt hơn OCSVM ở dormant_wakeup và off_hours về ROC-AUC. Điều này ủng hộ việc kết hợp mô hình và luật (mục 8).

4.5. Chi phí tính toán

Bước	Thời gian	Ghi chú
Làm sạch + đặc trưng 60 ngày	vài giờ	Bước ghép lịch sử 7 ngày chiếm phần lớn; 14 ngày tốn thêm khoảng 200s, gần gấp đôi RAM
Fit OCSVM `g01` / `nc1000`	5 s / 47 s	315 nghìn dòng train; `nc1000` dùng ma trận 315k × 1000 (> 2 GB, biến đổi theo khối)
Fit IF / LOF	12 s / 8 s	LOF HNSW đa luồng; LOF chính xác không khả thi ở quy mô này
Chấm 154 nghìn dòng	1–3 s mỗi mô hình	Đủ nhanh cho chấm theo lô hằng ngày
Lưới test (3 mô hình + 4 baseline × 5 seed + quét contamination)	khoảng 1 giờ	Phần lớn là 120 lần fit của quét contamination

5. Phân tích

5.1. Vì sao OCSVM vượt LOF

Nhiều bài so sánh xếp OCSVM ở nhóm kém. Kết quả ngược lại ở đây có ba nguyên nhân:

1. Loại bất thường quyết định thuật toán thắng. Các khảo sát lớn (ví dụ ADBench, NeurIPS 2022) cho thấy phương pháp toàn cục thắng với bất thường toàn cục, còn LOF mạnh với bất thường cục bộ. Năm trên sáu kịch bản ở đây là bất thường toàn cục.
2. Bằng chứng trực tiếp: LOF gần như mù với brute-force trong khi OCSVM đạt 0,99. Train có sẵn khoảng 600 dòng User thật giống brute-force. Nạn nhân tiêm có láng giềng gần nhất chính là các dòng này, mật độ tương đương nên LOF ≈ 1, tức "bình thường". OCSVM đo khoảng cách tới khối dữ liệu chính; 600 trên 315.000 dòng là vùng thưa nên bị coi là bất thường.
3. Tiền xử lý quyết định (mục 5.2).

5.2. Scaler: lựa chọn "an toàn" theo sách giáo khoa có thể sai

`RobustScaler` chia mỗi đặc trưng cho IQR của train. Trên dữ liệu UEBA, nhiều đặc trưng là tỷ lệ phần lớn bằng 0: đặc trưng có IQR rất nhỏ bị phóng đại (`failure_ratio` lên khoảng 1.400 đơn vị), còn đặc trưng có IQR = 0 giữ nguyên đơn vị gốc, trong đó có chính các đặc trưng tách nạn nhân tốt nhất. Mô hình dựa trên khoảng cách chỉ "thấy" vài đặc trưng bị phóng đại.

OCSVM, dev, 22 đặc trưng	Recall top-100/ngày	PR-AUC
RobustScaler	0,094	0,033
QuantileTransformer	0,174	0,073
StandardScaler	0,412	0,239

LOF thì ngược lại: RobustScaler tốt nhất (0,078 so với 0,014 PR-AUC), vì nó bắt mạnh vài kịch bản có đặc trưng bị phóng đại. Isolation Forest không bị ảnh hưởng vì không phụ thuộc thang đo.

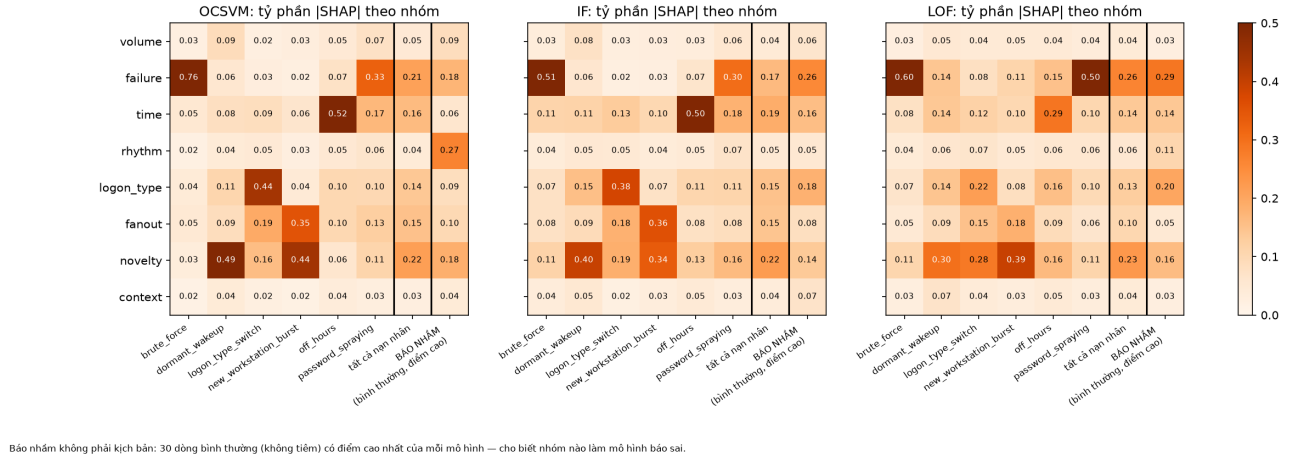
5.3. Chọn đặc trưng: từ 41 xuống 22

5.3.1. SHAP: mô hình dùng đúng tín hiệu

KernelSHAP trên một run dev, 180 nạn nhân (30 mỗi kịch bản) và 30 báo nhầm điểm cao nhất của mỗi mô hình. Tỷ phần |SHAP| được chuẩn hoá để so giữa ba mô hình.

Kịch bản	Nhóm đặc trưng chính (tỷ phần OCSVM / IF / LOF)
brute_force	failure 0,76 / 0,51 / 0,60
off_hours	time 0,52 / 0,50 / 0,29
logon_type_switch	logon_type 0,44 / 0,38 / 0,22
password_spraying	failure 0,33 / 0,30 / 0,50, sau đó là time
new_workstation_burst	novelty 0,44 / 0,34 / 0,39 và fanout 0,35 / 0,36 / 0,18
dormant_wakeup	novelty 0,49 / 0,40 / 0,30 (chủ yếu `days_since_last_activity`)
Báo nhầm(dòng bình thường điểm cao nhất)	OCSVM: rhythm 0,27; IF, LOF: failure

Mỗi kịch bản được giải thích bằng đúng nhóm đặc trưng thiết kế cho nó, nhất quán ở cả ba mô hình. Nhóm rhythm (nhịp giữa các sự kiện) chỉ chiếm khoảng nửa mức chia đều ở nạn nhân, nhưng chiếm 0,27 (gấp gần 3 lần mức chia đều) ở báo nhầm của OCSVM, gần như toàn bộ do `delta_t_cv`. LIME khớp kém ở từng dòng nên SHAP là căn cứ chính.



Hình 5.1. Tỷ phần |SHAP| theo nhóm (hàng) và kịch bản (cột), mỗi bảng một mô hình. Ô đậm gần như nằm trên một "đường chéo": mỗi kịch bản có đúng một nhóm nổi bật.

5.3.2. Ablation theo nhóm đặc trưng (đề cương 5.6)

Bỏ lần lượt từng nhóm, so với đủ 41 đặc trưng trên cùng run và seed (5 run dev). Riêng việc xáo thứ tự dòng train đã làm PR-AUC OCSVM lệch $-0{,}020 \pm 0{,}019$, nên thay đổi nhỏ hơn mức này coi như nhiễu.

Bỏ nhóm (còn lại)	ΔPR-AUC OCSVM	ΔPR-AUC IF	ΔPR-AUC LOF	Thay đổi rõ nhất theo kịch bản
failure (35)	$-0{,}056 \pm 0{,}018$	$-0{,}007$	$+0{,}002$	OCSVM brute_force $-0{,}57$ recall
novelty (36)	$-0{,}039 \pm 0{,}016$	$-0{,}018$	$-0{,}028$	OCSVM burst $-0{,}32$; LOF logon_type_switch $-0{,}50$
fanout (35)	$-0{,}014 \pm 0{,}003$	$-0{,}012$	$+0{,}002$	OCSVM burst $-0{,}14$
logon_type (35)	$-0{,}011 \pm 0{,}005$	$+0{,}004$	$-0{,}012$	logon_type_switch $-0{,}14$ đến $-0{,}28$
time (33)	$-0{,}002 \pm 0{,}003$	$+0{,}012$	$-0{,}012$	off_hours gần như không đổi
volume + context (35)	$+0{,}014 \pm 0{,}003$	$+0{,}038$	$+0{,}003$	không kịch bản nào tụt rõ
rhythm (37)	$+0{,}050 \pm 0{,}013$	$+0{,}019$	$+0{,}029$	không kịch bản nào tụt; brute_force $+0{,}13$ (OCSVM)
đối chứng xáo dòng (41)	*$-0{,}020 \pm 0{,}019$*	*$0{,}000$*	*$-0{,}005$*	

- failure và novelty là xương sống; bỏ chúng làm mất đúng kịch bản tương ứng.
- rhythm, volume, context có hại hoặc vô ích: bỏ đi làm cả ba mô hình tốt hơn.
- Bỏ nhóm time không làm off_hours tệ đi: các đặc trưng thời gian hiện có không giúp bắt kịch bản này.

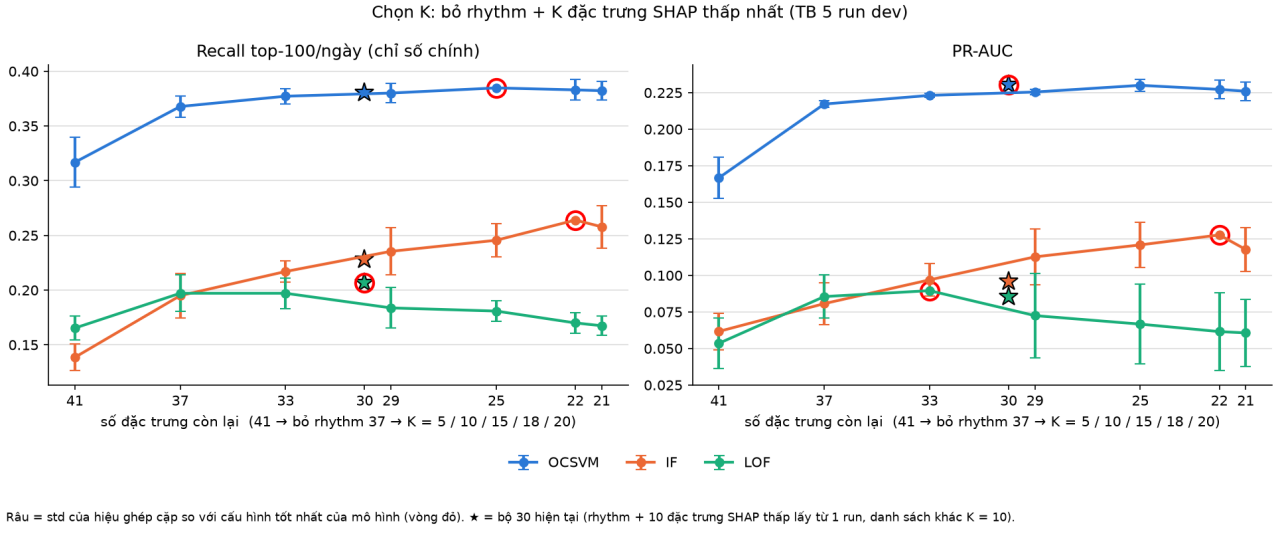
5.3.3. Chọn số đặc trưng bỏ (K)

Bộ 30 đặc trưng của tuần 5 bỏ nhóm rhythm và 10 đặc trưng SHAP thấp nhất. Con số 10 là số tròn: giá trị SHAP giảm đều, không có chỗ gãy (hạng 10 và hạng 11 chỉ chênh 0,0002). Vì vậy số đặc trưng cần bỏ được chọn lại bằng đường cong K trên dev, với quy tắc chọn đặt trước khi xem kết quả.

Xếp 41 đặc trưng theo tỷ phần |SHAP| cao nhất qua 3 mô hình $\times$ 3 run dev, rồi bỏ rhythm và K đặc trưng thấp nhất. K = 18 là mức cắt tự nhiên: đúng 18 đặc trưng có tỷ phần dưới mức chia đều 1/41 ở mọi mô hình, mọi run.

Còn lại	OCSVM	IF	LOF
41 (đủ)	0,317 ± 0,058	0,139 ± 0,011	0,165 ± 0,017
37 (bỏ rhythm, K = 0)	0,368 ± 0,057	0,195 ± 0,023	0,197 ± 0,014
33 (K = 5)	0,377 ± 0,059	0,217 ± 0,014	0,197 ± 0,018
30 (rhythm + 10 SHAP thấp, bộ của tuần 5)	0,380 ± 0,059	0,228 ± 0,017	0,207 ± 0,014
29 (K = 10)	0,380 ± 0,061	0,235 ± 0,022	0,184 ± 0,015
25 (K = 15)	0,385 ± 0,059	0,246 ± 0,019	0,181 ± 0,015
22 (K = 18, chốt)	0,383 ± 0,067	0,264 ± 0,012	0,170 ± 0,006
21 (K = 20)	0,382 ± 0,064	0,258 ± 0,021	0,167 ± 0,016

Recall top-100/ngày, TB ± std qua 5 run dev. Bộ 30 dùng xếp hạng SHAP của 1 run nên không trùng với K = 10.



Hình 5.2. Đường cong K trên dev (TB 5 run): trái là recall top-100/ngày, phải là PR-AUC. Vòng đỏ là K tốt nhất của từng mô hình, ngôi sao là bộ 30 của tuần 5.

- OCSVM phẳng từ 33 đến 22 đặc trưng; IF tăng tới 22 rồi dừng; LOF tốt nhất 30–33 rồi giảm.
- Không có K chung tốt nhất cho cả ba mô hình. K = 18 được chốt vì là mức cắt có căn cứ, OCSVM không đổi, IF tăng rõ, và K = 20 không tốt hơn ở mô hình nào. Đánh đổi là LOF kém trên dev (−0,037 ± 0,009).
- Kiểm tra trên test (bảng 5.1 của báo cáo tuần 5): so với bộ 30, K = 18 cho OCSVM −0,003 ± 0,005, IF +0,034 ± 0,013, LOF +0,028 ± 0,005 recall. LOF đảo chiều giữa dev và test, nên không dùng LOF để chọn K.
- Thiên lệch chọn: K = 18 được chốt sau khi đã xem kết quả test của 4 cấu hình K. Với OCSVM và IF, test chỉ xác nhận dev; với LOF việc chọn có tính đến test. Đây là ngoại lệ duy nhất với nguyên tắc "test không tham gia chọn".

Kết quả so với 41 đặc trưng ban đầu (test): recall top-100/ngày IF 0,16 → 0,24, LOF 0,13 → 0,18, OCSVM 0,32 → 0,34 (`g01`; `nc1000` 0,37). IF được lợi nhiều nhất vì mỗi chiều yếu làm loãng cây.

5.4. Cửa sổ lịch sử 7 / 14 / 21 ngày

Kéo dài cửa sổ của các đặc trưng lịch sử không cải thiện kết quả: recall top-100/ngày OCSVM 0,317 / 0,318 / 0,294; LOF 0,121 / 0,034 / 0,003. Cửa sổ dài chỉ giúp spraying (ROC-AUC 0,69 → 0,77) nhưng làm hại dormant_wakeup và tốn khoảng 3 lần thời gian. Giữ 7 ngày.

5.5. Độ ổn định

Qua seed. Std của recall top-100/ngày: IF 0,016, LOF 0,006, OCSVM `g01` 0,070. Std gồm cả biến động dữ liệu tiêm và mô hình. Riêng OCSVM còn nhạy với thứ tự dòng train (SGD), nên dòng được sắp cố định trước khi fit.

Chọn cấu hình OCSVM trên dev không lặp lại trên test. Trên dev, `g01` thắng `nc1000` với mức chênh nhỏ hơn 1 std. Trên test:

OCSVM, test	PR-AUC từng run	TB ± std	Recall top-100/ngày
`g01` (300 thành phần Nystroem, γ = 0,1)	0,19 / 0,20 / 0,11 / 0,15 / 0,24	0,175 ± 0,048	0,336 ± 0,070
`nc1000` (1.000 thành phần, γ = scale)	0,19 / 0,19 / 0,17 / 0,20 / 0,20	0,189 ± 0,013	0,373 ± 0,022

Chênh lệch trung bình nằm trong 1 std, nhưng `g01` dao động gấp 3–4 lần. Báo cáo vẫn dùng `g01` vì đó là cấu hình chọn trên dev; đổi sau khi xem test là chọn mô hình bằng tập test. Khi triển khai nên dùng `nc1000` (mục 8), và quy tắc chọn về sau nên xét cả độ lệch chuẩn.

LOF không tất định. Chỉ mục HNSW dựng đa luồng (`n_jobs: -1`) đổi thứ tự chèn mỗi lần fit:

- Trung bình 5 run lặp lại: hai lần chạy lưới test cho recall 0,176 và 0,175, PR-AUC 0,070 và 0,071.
- Nhưng từng lần fit lệch nhiều: PR-AUC của run seed 42 là 0,072 / 0,075 / 0,079 / 0,083 qua bốn lần fit (lệch tới khoảng 15%), và top 50 cảnh báo của hai lần fit trên log gốc chỉ trùng 6/50.
- Hệ quả: danh sách cảnh báo của LOF không tái lập được; đặt `n_jobs: 1` thì tất định nhưng fit chậm hơn nhiều.

5.6. Mức lạc quan của kết quả

- Do chọn cấu hình trên dev: nhỏ. Dev nằm trong train; cấu hình 41 đặc trưng, PR-AUC test bằng hoặc cao hơn dev ở cả ba mô hình. Với 22 đặc trưng, PR-AUC test thấp hơn dev (OCSVM 0,175 so với 0,248), một phần do dev và test khác nhau về độ dài train và số ngày chấm.
- Do nhãn tổng hợp: lớn và không đo được. Tấn công tiêm là bản sao sự kiện thật nhưng theo khuôn cố định; tấn công thật đa dạng và kín đáo hơn. Mọi con số trong báo cáo là cận trên.

6. Đánh giá định tính: review thủ công top 50

6.1. Thiết lập

Trên log gốc, không tiêm, mô hình fit ngày 1–42 và chấm ngày 43–60 (phân khúc User, seed 42). Lấy 50 cảnh báo điểm cao nhất của mỗi mô hình, hợp lại, xáo thứ tự và làm mù. Mỗi cảnh báo có một "thẻ" gồm đặc trưng lệch nhất so với train, ngữ cảnh tài khoản và tóm tắt log thô của ngày đó. Ba nhãn theo đề cương 5.5: a đáng ngờ, cần điều tra; b bất thường nhưng giải thích được; c nhiễu.

6.2. Trạng thái

Bộ	Mô hình	Số cảnh báo	Người chấm	Trạng thái
`review_top50_22f`	22 đặc trưng, OCSVM `g01`	148	2 thực tập sinh, độc lập	Chưa chấm

> Một phần Mục này chưa hoàn chỉnh. Đề cương yêu cầu hai TTS chấm độc lập rồi đo đồng thuận (Cohen's κ). Khi có nhãn của hai người, cần bổ sung bảng phân loại a/b/c theo mô hình, κ và nhận xét.

7. Hạn chế đã biết

1. Nhãn tổng hợp cho cận trên lạc quan. Tấn công thật đa dạng và kín đáo hơn khuôn tiêm; dòng nhãn 0 cũng chưa chắc lành tính. Không có nhãn tấn công thật để hiệu chỉnh.
2. Giả định train "sạch" không hoàn toàn đúng. Train có khoảng 600 dòng giống brute-force thật; chính chúng làm LOF thất bại.
3. Chỉ đánh giá phân khúc User. Tài khoản Machine không được tiêm; Admin, Service, System quá ít để học máy.
4. Ba kịch bản bị bắt kém: password_spraying, off_hours, dormant_wakeup (recall top-100/ngày $\leq$ 0,23 / 0,08 / 0,17, số của 41 đặc trưng).
5. Thuật toán xấp xỉ. LOF dùng láng giềng xấp xỉ HNSW, OCSVM dùng nhân RBF xấp xỉ Nystroem + SGD, thay cho bản chính xác của scikit-learn. Kết luận về "LOF" và "OCSVM" là kết luận về các bản xấp xỉ này.
6. LOF không tất định ở mức từng lần fit (mục 5.5); danh sách cảnh báo LOF không tái lập được.
7. OCSVM `g01` kém ổn định trên test; quy tắc chọn chỉ dựa trên trung bình.
8. Thiên lệch chọn ở bước chọn K (mục 5.3.3): K = 18 được chốt sau khi xem test.
9. Train dùng chung cho mọi run: chỉ ngày chấm bị tiêm, nên độ lệch chuẩn không gồm biến động của train.
10. Ngưỡng theo phân vị train không giữ ngân sách khi hành vi trôi: tỷ lệ cảnh báo thực tế cao hơn contamination 1,4–2,5 lần.
11. Đổi LogonType chỉ có 5–8 nạn nhân/run theo đúng chiều 2 $\rightarrow$ 3/5 của đề cương.
12. Review thủ công của bộ 22 đặc trưng chưa có nhãn của hai người, nên báo cáo chưa có kết quả định tính trên log thật.
13. Tái lập trên máy sạch chưa được thực hiện (mục 9).

8. Khuyến nghị triển khai

8.1. Kiến trúc phân luồng theo loại tài khoản

Luồng	Đối tượng	Phương pháp
Học máy	Tài khoản User (khoảng 45% dòng)	OCSVM làm điểm chính, IF làm mô hình phụ
Luật dò/rải mật khẩu	Mọi tài khoản, theo máy nguồn	Đếm số tài khoản thất bại theo `Source` trong cửa sổ ngắn; luật số lần thất bại theo tài khoản
Luật và danh sách trắng	Admin, Service, System	Mỗi tài khoản dịch vụ chỉ được phép một tập cố định; vi phạm là cảnh báo mức cao
Theo dõi riêng (chưa đánh giá)	Machine	Cần đánh giá riêng trước khi đưa vào học máy

8.2. Cấu hình mô hình

1. OCSVM StandardScaler, 1.000 thành phần Nystroem (`nc1000`), 22 đặc trưng. Lý do: recall top-100/ngày 0,373 $\pm$ 0,022, ổn định nhất trong các cấu hình OCSVM tốt; `g01` nhanh hơn 9 lần nhưng dao động lớn (mục 5.5). Nếu ràng buộc thời gian fit quan trọng hơn, dùng `g01` và theo dõi chặt chỉ số hằng ngày.
2. IF 300 cây, max_samples 1024 làm mô hình phụ: tốt hơn OCSVM ở dormant_wakeup và off_hours về ROC-AUC.
3. Không dùng LOF ở cấu hình hiện tại: kém nhất trên nhãn tổng hợp và không tất định.
4. Kiểm tra thang đo sau tiền xử lý mỗi khi đổi bộ đặc trưng (mục 5.2).

8.3. Ngân sách cảnh báo và chi phí vận hành

Ngân sách	Contamination (khoảng)	Recall (OCSVM)	Cảnh báo đúng / cảnh báo	Thời gian analyst (giả định 5 phút/cảnh báo)
khoảng 40 cảnh báo/ngày	0,2%	0,16	khoảng 1/3	khoảng 3,3 giờ/ngày
khoảng 100 cảnh báo/ngày	0,5%	0,31	khoảng 1/4	khoảng 8 giờ/ngày (1 analyst)
khoảng 280 cảnh báo/ngày	2%	0,52	khoảng 1/6	khoảng 23 giờ/ngày (3 analyst)

Khuyến nghị khởi đầu ở khoảng 100 cảnh báo/ngày (một analyst toàn thời gian) và đặt ngân sách theo số cảnh báo/ngày thay vì contamination. Thời gian 5 phút/cảnh báo là giả định, chưa đo thực tế; recall là cận trên.

8.4. Vận hành và theo dõi

1. Huấn luyện lại định kỳ (gợi ý hằng tuần) trên cửa sổ trượt; theo dõi tỷ lệ cảnh báo và phân bố điểm theo ngày để phát hiện trôi dữ liệu.
2. Giải thích từng cảnh báo bằng SHAP theo nhóm đặc trưng giúp analyst biết cảnh báo thuộc loại nào (thất bại, nguồn mới, loại logon, . . .).
3. Ưu tiên cải tiến tiếp theo: đặc trưng fan-out theo máy nguồn (spraying), đặc trưng giờ so với lịch sử cá nhân (off_hours), đánh giá trên tấn công thật (ví dụ `redteam.txt` của LANL).

9. Tái lập và vận hành

9.1. Môi trường

Cài theo README mục 2 (`pip install -r requirements.txt`, `pip install -e .`). Dữ liệu interim (khoảng 22 GB) tải từ Google Drive (README mục 6). Mọi tham số nằm trong `configs/`.

Phiên bản thư viện: `requirements.txt` chỉ ghi phiên bản tối thiểu. Để chạy lại đúng như báo cáo, cài đúng các phiên bản sau (ghi trong manifest của lưới test):

Thư viện	Phiên bản
Python	3.14.6
numpy	2.5.3
pandas	3.0.5
polars	1.44.2
pyarrow	24.0.0
scipy	1.18.1
scikit-learn	1.9.1
joblib	1.6.0

Kiểm tra dữ liệu sau khi tải: `python scripts/validate_interim.py` phải báo 120 file hợp lệ, gồm 929.103.392 dòng 4624 và 13.823.040 dòng 4625.

9.2. Một lệnh

```
python scripts/reproduce.py        # đặc trưng, hồ sơ train, baseline, tiêm 5 run test, lưới 22 đặc
↪  trưng, đối chiếu số
```

Kết quả được so với bảng 4.2, đạt khi sai lệch tương đối $\leq 1\%$ (LOF $\leq 5\%$). Mọi bước ngẫu nhiên có seed cố định.

9.3. Từng bước

```
python -m pytest tests -q                                                    # 258 test
python main.py --stage features                                              # log -> ma trận đặc trưng
python main.py --stage inject --block test                                   # tiêm một run
python scripts/evaluation/run_grid.py --config configs/benchmark_grid_dev_22f.yaml    # bảng 4.1,
↪  khoảng 15 phút
python scripts/evaluation/run_grid.py --config configs/benchmark_grid_test_22f.yaml   # bảng 4.2-4.4,
↪  khoảng 1 giờ
python scripts/evaluation/explain_models.py                                  # SHAP/LIME
python scripts/evaluation/feature_ablation.py [--mode kcurve]                # ablation nhóm / chọn K
python scripts/evaluation/build_review_set.py --config configs/benchmark_grid_test_22f.yaml --out
↪  experiments/analysis/review_top50_22f
python scripts/evaluation/review_agreement.py                                # Cohen's kappa
python scripts/evaluation/train_demo_models.py                               # lưu 3 mô hình cho demo
```

9.4. Mô hình cho demo

Script `train_demo_models.py` fit 3 mô hình của bảng 4.2 trên log gốc ngày 1–42 và lưu pipeline (điền thiếu + scaler + mô hình). Nạp lại cho PR-AUC khớp với lưới, trừ LOF (không tất định).

9.5. Trạng thái kiểm chứng

- `pytest`: 258 test pass; bản clone sạch chạy được test mà không cần dữ liệu ngoài Git (trừ 2 test).
- Chưa chạy `reproduce.py` trên một máy sạch do người thứ ba thực hiện (tiêu chí 4.1 #1).
- Chưa có file khoá phiên bản (`pip freeze`); bảng ở mục 9.1 thiếu `pyod`, `hnswlib`, `shap`, `lime`.
- Chưa có mã băm (sha256) của dữ liệu interim. Hiện chỉ kiểm được bằng số file và số dòng.

10. Đối chiếu tiêu chí hoàn thành (đề cương mục 4.1)

#	Hạng mục	Trạng thái	Bằng chứng / việc còn lại
1	Tính tái lập	Một phần	Có lệnh duy nhất và seed cố định (mục 9). Chưa chạy trên máy sạch; LOF chỉ tái lập ở mức trung bình
2	Pipeline dữ liệu	Đạt	Mục 2.3: quyết định làm sạch kèm lý do, có ghi log
3	Bộ đặc trưng	Đạt	41 core (22 dùng trong benchmark); tên, công thức, kiểu và ý nghĩa của 22 đặc trưng ở Phụ lục A, đủ 41 trong schema; unit test
4	Bảng benchmark	Đạt	Bảng 4.1: 3 mô hình × 3–6 cấu hình × 5 seed, mean ± std; bảng 4.2 trên test
5	Bộ chỉ số	Đạt	Bảng 4.2: PR-AUC, ROC-AUC, P@10/50/100, recall tại ngân sách, tỷ lệ cảnh báo, thời gian fit/chấm
6	Baseline	Đạt	Ngẫu nhiên và luật số lần thất bại (cùng z-score, luật 6 điều kiện) trong mọi bảng
7	Phân tích lỗi	Một phần	Phân tích theo kịch bản xong (mục 4.4). Review top 50 bộ 22 đặc trưng chờ hai người chấm và κ (mục 6)
8	Báo cáo kỹ thuật	Đạt	Tài liệu này
9	Chất lượng mã	Một phần	Cấu trúc thư mục, cấu hình tách rời, 258 test. Chờ review chéo và mentor duyệt
10	Nghiệm thu	Chưa	Chờ buổi demo; mô hình demo đã sẵn sàng (mục 9.4)

Phụ lục A: 22 đặc trưng của benchmark cuối

Một dòng là một tài khoản trong một ngày. "7 ngày trước" là 7 ngày lịch t−7 … t−1. Cột "NULL khi" ghi trường hợp đặc trưng không có giá trị; khi đó mô hình điền bằng trung vị của train (mục 3.2).

Nhóm	Đặc trưng	Kiểu	Công thức	Ý nghĩa hành vi	NULL khi
failure	`failure_ratio`	số thực	số lần thất bại (4625) / tổng số lần đăng nhập trong ngày	Tỷ lệ đăng nhập thất bại trong ngày	–
failure	`failure_locked_out_share`	số thực	số lần thất bại có lý do "account locked out" / số lần thất bại	Tỷ phần thất bại do tài khoản bị khoá (dấu hiệu dò mật khẩu)	không có thất bại nào
failure	`delta_mean_share_fail_7d`	số thực	tỷ lệ thất bại hôm nay − trung bình tỷ lệ thất bại của 7 ngày trước	Tỷ lệ thất bại tăng bất thường so với chính tài khoản	7 ngày đầu dữ liệu, hoặc 7 ngày trước không hoạt động
failure	`novelty_fail_host_7d`	số thực	số máy đích (`LogHost`) bị đăng nhập thất bại hôm nay không có trong 7 ngày trước	Thất bại ở những máy chưa từng thử	–
failure	`novelty_fail_failreason_7d`	số thực	số lý do thất bại hôm nay không có trong 7 ngày trước	Kiểu thất bại mới (ví dụ sai mật khẩu, tài khoản không tồn tại)	–
time	`activity_peak_hour_cos`	số thực	$\cos(2\pi \cdot h / 24)$, với h là giờ có nhiều sự kiện nhất trong ngày	Giờ cao điểm trong ngày, mã hoá vòng tròn để 23h và 0h nằm cạnh nhau	–
time	`hour_entropy`	số thực	entropy của phân bố sự kiện theo giờ / ln(số giờ có sự kiện)	Hoạt động dồn vào ít giờ (gần 0) hay trải đều (gần 1)	–
time	`dist_shift_hour_7d`	số thực	$\frac{1}{2} \cdot \Sigma \lvert$tỷ phần sự kiện ở giờ h hôm nay − tỷ phần ở giờ h của 7 ngày trước$\rvert$	Phân bố giờ hoạt động thay đổi bao nhiêu so với tuần trước (0 là y hệt, 1 là khác hẳn)	chưa có lịch sử 7 ngày
time	`jaccard_hour_7d`	số thực	$\lvert$giờ hoạt động hôm nay ∩ của 7 ngày trước$\rvert$ / $\lvert$hợp của hai tập$\rvert$	Độ trùng tập giờ hoạt động với tuần trước	chưa có lịch sử 7 ngày
time	`delta_mean_share_night_7d`	số thực	tỷ lệ sự kiện ngoài giờ (18h–7h) hôm nay − trung bình của 7 ngày trước	Hoạt động dời sang ban đêm so với chính tài khoản	7 ngày đầu dữ liệu
logon_type	`interactive_ratio`	số thực	số lần đăng nhập loại 2 (tương tác) / tổng số lần đăng nhập	Tỷ lệ đăng nhập trực tiếp tại máy	–
logon_type	`ntlm_ratio`	số thực	số lần dùng gói xác thực NTLM / tổng số lần đăng nhập	Tỷ lệ xác thực NTLM (giao thức cũ, hay gặp trong tấn công)	–
logon_type	`dist_shift_logontype_7d`	số thực	như `dist_shift_hour_7d`, nhưng theo loại đăng nhập thay cho giờ	Cơ cấu loại đăng nhập thay đổi so với tuần trước	chưa có lịch sử 7 ngày
logon_type	`novelty_logontype_7d`	số thực	số loại đăng nhập hôm nay không có trong 7 ngày trước	Dùng loại đăng nhập chưa từng dùng	–
fanout	`distinct_sources_count`	số nguyên	số máy nguồn (`Source`) khác nhau trong ngày	Đăng nhập từ bao nhiêu máy	–
fanout	`delta_mean_distinct_source_7d`	số thực	log(số máy nguồn hôm nay) − trung bình của 7 ngày trước	Số máy nguồn tăng bất thường so với chính tài khoản	7 ngày đầu dữ liệu

Nhóm	Đặc trưng	Kiểu	Công thức	Ý nghĩa hành vi	NULL khi
fanout	`peer_z_distinct_source`	số thực	(log số máy nguồn − trung vị của các tài khoản cùng loại trong ngày) / (IQR / 1,349), chặn trong ±10	Số máy nguồn cao bất thường so với các tài khoản cùng loại	–
novelty	`new_source_count_7d`	số nguyên	số máy nguồn hôm nay không có trong 7 ngày trước	Đăng nhập từ máy mới	–
novelty	`days_since_last_activity`	số nguyên	t − ngày hoạt động gần nhất trước t	Tài khoản nghỉ bao lâu trước hôm nay	lần hoạt động đầu tiên của tài khoản
novelty	`jaccard_source_7d`	số thực	|máy nguồn hôm nay ∩ của 7 ngày trước| / |hợp của hai tập|	Độ trùng tập máy nguồn với tuần trước	chưa có lịch sử 7 ngày
novelty	`novelty_share_pair_7d`	số thực	số sự kiện qua cặp (máy nguồn, máy đích) mới / tổng số sự kiện	Tỷ phần đăng nhập đi theo đường mới	không có sự kiện có đủ nguồn và đích
volume	`delta_mean_count_7d`	số thực	log(số sự kiện hôm nay) − trung bình của 7 ngày trước	Khối lượng tăng hoặc giảm bất thường so với chính tài khoản	7 ngày đầu dữ liệu

19 đặc trưng bị bỏ: 4 rhythm (`interarrival_dt_mean`, `delta_t_cv`, `same_second_share`, `is_single_event`) và `activity_peak_hour_sin`, `log_distinct_hosts`, `missing_source_ratio`, `dst_host_entropy`, `log_total_logons`, `custom_proc_share`, `delta_mean_distinct_pair_7d`, `new_host_count_7d`, `peer_z_share_night`, `rare_logon_type_count_log`, `remote_logon_ratio`, `volume_robust_z_7d`, `off_hours_ratio`, `delta_mean_distinct_fail_host_7d`, `jaccard_logontype_7d`. Công thức đầy đủ của cả 41 đặc trưng: `configs/feature_schema.yaml` và README mục 4.

Phụ lục B: bản đồ số liệu

Nội dung	Đường dẫn
Đề cương	`docs/Tong quan de tai ueba.md`
Từ điển dữ liệu, quyết định làm sạch	`docs/data_dictionary.md`, `docs/nhat_ky_trien_khai_pipeline.md`
Phân loại tài khoản	`docs/reports/thong_ke_loai_tai_khoan.md`
Bộ đặc trưng v4	`docs/reports/bao_cao_bo_dac_trung_v4.md`, `configs/feature_schema.yaml`
Bộ tiêm	`docs/huong_dan_tiem_bat_thuong_tuan4.md`, `configs/injection.yaml`, `src/injection/`
Baseline	`reports/week4/baselines_report.md`, `configs/baselines.yaml`
Lưới dev / test (22 đặc trưng)	`experiments/grid/dev_22f/`, `experiments/grid/test_22f/`, `experiments/grid/test_22f_nc1000/`
Lưới 41 đặc trưng (tuần 4)	`experiments/grid/dev/`, `experiments/grid/test/`, `reports/week4/tom_tat_tuan4.md`
Kịch bản, SHAP, ablation, chọn K	`reports/week5/` (`bao_cao_tuan5.md` và các file chi tiết), `experiments/analysis/`, `experiments/ablation/`
Review top 50	`experiments/analysis/review_top50_22f/` (chưa chấm)
Nhật ký thí nghiệm	`experiments/logs/experiment_log.csv`